



**NITTE**  
(Deemed to be University)

**NMAM INSTITUTE  
OF TECHNOLOGY**

Nitte (DU) established under Section 3 of UGC Act 1956 | Accredited with 'A+' Grade by NAAC

---

**Department of Computer Science & Engineering**

---

Report on CD Mini Project

## Reverse a String

Course Code: CS3602-1

Course Name: Compiler Design Lab

Semester: VI SEM

Section: D

**Submitted To:**

Dr. Asmitha Poojari

Assistant Professor Grade - III

Department of Computer Science and Engineering

**Submitted By:**

Sheethal K Shetty (NNM23CS182)

Sindhushree Shetty (NNM23CS202)

Sunidhi Girish (NNM23CS209)

**Date of submission:**

17/04/2026



**NITTE**  
(Deemed to be University)

**NMAM INSTITUTE  
OF TECHNOLOGY**

Nitte (DU) established under Section 3 of UGC Act 1956 | Accredited with 'A+' Grade by NAAC

---

## CERTIFICATE

*This is to certify that the “Mini-Project report” submitted by **Ms. Sheethal K Shetty, Ms. Sindhushree Shetty and Ms. Sunidhi Girish** bearing USN **NNM23CS182, NNM23CS202, NNM23CS209** of VI SEM B.Tech., Bonafide students of NMAM Institute of Technology, Nitte, has successfully completed CD Mini-Project on “Design a Compiler for a program to find largest number using if statement” during 2025-2026 in **Computer Science and Engineering** at NMAM Institute of Technology, Nitte.*

---

*Signature of the Course Instructor*

---

*Signature of the HOD*

## ABSTRACT

---

A compiler is an essential system software that translates high-level programming languages into machine-understandable code. This project focuses on the design and implementation of a compiler using Python, demonstrating the fundamental phases involved in compilation. The system includes lexical analysis, syntax analysis, and parsing. The lexical analyser scans the input program and generates tokens, while the syntax analyser verifies the correctness of the program using a context-free grammar. The project also involves preprocessing steps such as removal of left recursion and left factoring to make the grammar suitable for parsing. Canonical LR (CLR (1)) parsing technique is implemented to perform syntax analysis. FIRST and FOLLOW sets are computed, and CLR states along with ACTION and GOTO tables are constructed. These tables are used by the parser to validate the given input string. The project provides a practical understanding of compiler design concepts and demonstrates how different phases of a compiler work together to analyse and validate a program efficiently.

# TABLE OF CONTENTS

---

**Title Page ..... 1**

**Certificate..... 2**

**Abstract..... 3**

**Table of Contents ..... 4**

**Introduction ..... 1-3**

**Problem Statement ..... 4**

**Design ..... 5-6**

**Requirements ..... 7**

**Implementation Details ..... 8-17**

**Results ..... 18-20**

**Conclusion..... 21**

**References..... 22**

# INTRODUCTION

## **Compiler:**

A compiler is a system software that translates a program written in a high-level programming language into an equivalent program in a target language, usually machine code. It plays a crucial role in the execution of programs by converting human-readable instructions into a form that can be understood and executed by a computer. The process of compilation involves analyzing the source code, detecting errors, and generating efficient machine-level instructions. Compilers not only translate the code but also ensure that the program follows the syntax and semantic rules of the programming language.

## **Key Components:**

A source program is written using a high-level programming language that consists of various constructs such as variables, data types, operators, and control statements. The compiler takes this source program as input and processes it through multiple stages to generate the corresponding machine-level output. During this transformation, the logical structure and meaning of the program are preserved. The output generated by the compiler is either executable code or an intermediate representation that can later be converted into machine code.

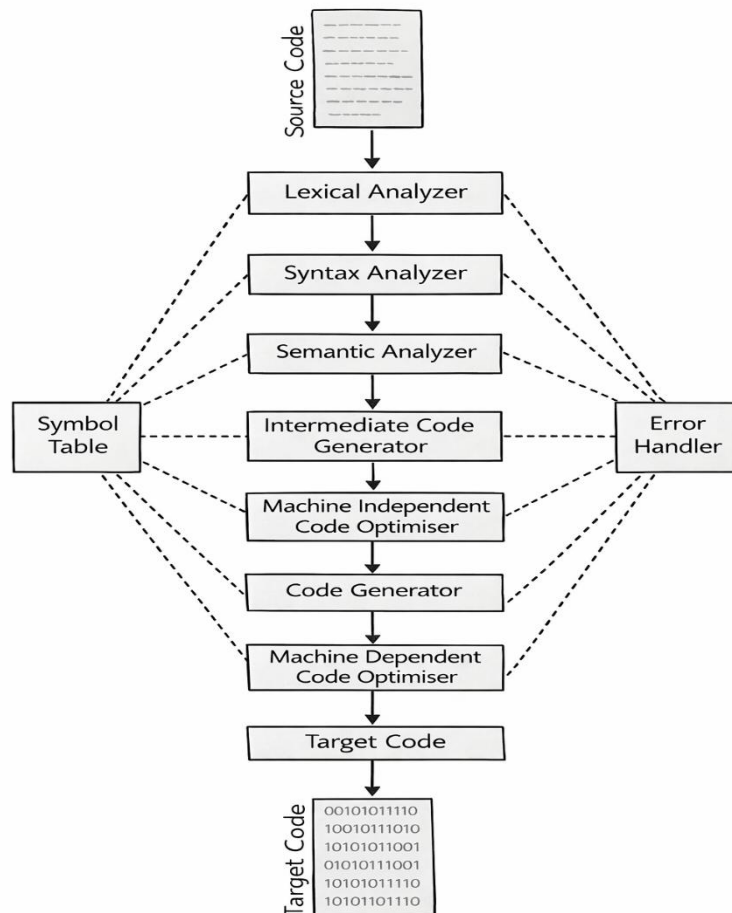
## **Error Handling:**

Error handling is an important aspect of compiler design. During the compilation process, errors may occur due to incorrect syntax, undeclared variables, or invalid operations. The compiler identifies these errors at different stages and provides appropriate error messages to the user. These messages help in identifying the location and type of error, making it easier to correct them. Effective error handling improves the reliability of the program and ensures that only correct and meaningful programs are executed.

## **Phases of Compilation:**

The compilation process is divided into several phases, each responsible for a specific task in transforming the source code into machine code. These phases include lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, and code

generation. Each phase takes the output of the previous phase as input and performs its designated function. This step-by-step process ensures that the program is systematically analyzed and converted into an executable form.



**Figure 1: Phases of Compiler**

## Lexical Analysis:

Lexical analysis is the first phase of the compiler. In this phase, the input program is scanned character by character and grouped into meaningful units called tokens. Tokens include keywords, identifiers, constants, operators, and punctuation symbols. The lexical analysis phase removes unnecessary spaces, comments, and formatting, making the input suitable for further processing. It plays a vital role in simplifying the input for the syntax analysis phase.

### **Syntax Analysis:**

Syntax analysis, also known as parsing, is the process of checking whether the sequence of tokens generated by the lexical analysis phase follows the grammar rules of the programming language. It constructs a parse tree that represents the hierarchical structure of the program. This phase ensures that the program is syntactically correct and helps in detecting errors related to the structure of the program.

### **Semantic Analysis:**

Semantic analysis focuses on checking the meaning of the program. It verifies whether the operations performed in the program are valid and meaningful. This includes checking type compatibility, variable declarations, scope rules, and function usage. Even if a program is syntactically correct, it may still contain semantic errors, which are identified during this phase.

### **Intermediate Code Generation:**

In this phase, the compiler generates an intermediate representation of the source program. This intermediate code is independent of both the source and target languages and acts as a bridge between them. It simplifies the process of optimization and code generation by providing a uniform representation of the program.

### **Code Optimization:**

Code optimization improves the efficiency of the intermediate code by removing redundant computations and simplifying expressions. The goal of optimization is to enhance the performance of the program without altering its output. This phase ensures that the final generated code uses fewer resources and executes faster.

### **Code Generation:**

Code generation is the final phase of a compiler, where optimized intermediate code is converted into machine code that can be executed by the computer. This phase ensures efficient mapping of the program to the hardware. In this project, a compiler is implemented using Python to demonstrate phases such as lexical analysis, syntax analysis, and parsing. The Canonical LR (CLR(1)) parsing technique is used to construct parsing tables and validate the input program based on the defined grammar, providing a practical understanding of compiler design concepts.

## PROBLEM STATEMENT

```
int main ( )  
begin  
    int n , re = 0 , rem ;  
    while ( expr )  
    begin  
        expr = expr + expr ;  
    end  
end
```



# DESIGN

## **Lexical Analysis:**

The initial stage of the compiler, lexical analysis, is responsible for processing the input source code obtained from the language pre-processors. It works as a scanner that reads the source code character by character and converts it into a sequence of tokens. While performing this process, it removes unnecessary elements such as whitespaces and comments. If any invalid token is encountered, the lexical analyzer generates an error message. The tokens generated are passed to the syntax analyzer for further processing. The primary objective of lexical analysis is to simplify the input and improve the efficiency of the subsequent phases.

## **Token:**

A token is a sequence of characters treated as a single logical unit during lexical analysis. Tokens represent meaningful elements of a program such as keywords, identifiers, operators, constants, and special symbols. Each token is defined by a pattern and is recognized by the lexical analyzer. Tokens play a crucial role in breaking down the input program into manageable pieces for syntax analysis.

## **Pattern:**

Patterns are rules associated with tokens that define the structure of strings in the input that correspond to a particular token. These rules are used to identify and classify lexemes in the source program. Patterns are generally defined using regular expressions and help in distinguishing different categories of tokens effectively.

## **Lexeme:**

A lexeme is the actual sequence of characters in the source program that matches the pattern of a token. It is the smallest unit in the program that carries meaning. For example, variable names, constants, and keywords are all lexemes. Each lexeme is associated with a token during lexical analysis.

## Syntax Analysis:

Syntax analysis is the phase in which the sequence of tokens generated by the lexical analyzer is checked against the grammar rules of the programming language. It ensures that the input program follows the correct structure and forms a valid sequence. The syntax analyzer constructs a parse tree that represents the hierarchical structure of the program. If any syntax errors are detected, appropriate error messages are generated to guide the user.

## Parser:

The parser, also known as the syntax analyzer, is a crucial component of the compiler. It takes the token stream as input and verifies whether it conforms to the grammar of the language. The parser constructs a parse tree and ensures the correctness of the program structure. In this project, the parser is implemented using the Canonical LR (CLR (1)) technique. It constructs CLR states and generates ACTION and GOTO tables, which are used to validate the input string and determine whether it is accepted or rejected.



**Figure 2:** Working of /lexical and Syntax Analyzer

# REQUIREMENTS

## Hardware Requirements:

- Processor: A modern multi-core processor (e.g., Intel Core i5 or equivalent).
- RAM: At least 8 GB RAM for efficient compilation and execution.
- Storage: Adequate storage space for development environment and project files.

## Software Requirements:

- Operating System: Windows, macOS, or Linux.
- Programming Language: Python.
- Python Libraries: The code uses the re (regular expressions) library and PrettyTable for tabular output.
- Installation Commands:
  - `pip install regex`
  - `pip install prettytable`
  - `pip install ply`
- Text Editor or IDE: Visual Studio Code, PyCharm, Sublime Text, or Atom.
- File Input: Ensure that the code has access to the input file specified in the program.
- Command Line or Terminal: Required to run Python scripts.
- Internet Connection: May be required to install Python libraries using pip.

## Additional Considerations:

- Python Version: Python 3.x

## IMPLEMENTATION DETAILS

Implementing a compiler involves several distinct stages, including lexical analysis, syntax analysis, and parsing. Below is a simplified Python implementation for the given problem statement.

```
lexer.py > ...
1  # lexer.py
2
3  keywords = {
4      "int": "DATATYPE",
5      "main": "MAIN",
6      "begin": "BEGIN",
7      "end": "END",
8      "while": "WHILE"
9  }
10
11  symbols = {
12      "(": "LPAR",
13      ")": "RPAR",
14      ",": "COMMA",
15      "=": "EQ",
16      "+": "PLUS",
17      ";": "SEMI"
18  }
19
20
21  def tokenize(code):
22
23      tokens = []
24
25      # Add spacing
26      code = code.replace("(", " ( ")
27      code = code.replace(")", " ) ")
28      code = code.replace(";", " ; ")
29      code = code.replace("=", " = ")
30      code = code.replace("+", " + ")
31      code = code.replace(",", " , ")
32
33      words = code.split()
34
35      for word in words:
36
37          if word in keywords:
38              tokens.append(keywords[word])
39
40          elif word in symbols:
41              tokens.append(symbols[word])
42
43          elif word.isdigit():
44              tokens.append("NUMBER")
45
46          else:
47              tokens.append("VAR")
48
49      tokens.append("$")
50
51      return tokens
52
```

Figure 3: Code to generate Tokens

```

cfg.py > ...
1  # cfg.py
2
3  grammar = {
4
5  "S": [
6  ["DATATYPE", "MAIN", "LPAR", "RPAR",
7  | "BEGIN", "DECL", "STMT", "END"]
8  ],
9
10 "DECL": [
11 ["DATATYPE", "VAR", "COMMA",
12 | "VAR", "EQ", "NUMBER",
13 | "COMMA", "VAR", "SEMI"]
14 ],
15
16 "STMT": [
17 ["WHILE", "LPAR", "EXPR",
18 | "RPAR", "BEGIN",
19 | "ASSIGN", "END"]
20 ],
21
22 "ASSIGN": [
23 ["VAR", "EQ", "EXPR", "SEMI"]
24 ],
25
26 # FIXED EXPR (No  $\epsilon$  now)
27

```

```

25
26 # FIXED EXPR (No  $\epsilon$  now)
27
28 "EXPR": [
29 ["VAR"],
30 ["VAR", "PLUS", "VAR"]
31 ]
32
33 }
34
35 start_symbol = "S"
36
37
38 if __name__ == "__main__":
39
40     print("CFG Grammar:\n")
41
42     for nt in grammar:
43         print(nt, "->", grammar[nt])

```

**Figure 4:** Code to define Grammar (CFG)

```

first.py > ...
1  # first.py
2
3  from cfg import grammar
4  from prettytable import PrettyTable
5
6  FIRST = {}
7
8  # Initialize FIRST sets
9  for nt in grammar:
10     FIRST[nt] = set()
11
12
13  def compute_first():
14
15     changed = True
16
17     while changed:
18
19         changed = False
20
21         for nt in grammar:
22
23             for production in grammar[nt]:
24
25                 before = FIRST[nt].copy()
26
27                 for symbol in production:
28
29                     # Terminal
30                     if symbol not in grammar:
31
32                         FIRST[nt].add(symbol)
33                         break
34
35                     # Non-terminal
36                     else:
37
38                         FIRST[nt] |= FIRST[symbol] - {"ε"}
39
40                         if "ε" not in FIRST[symbol]:
41                             break
42
43                 else:
44                     FIRST[nt].add("ε")
45
46                 if before != FIRST[nt]:
47                     changed = True
48
49
50  compute_first()
51

```

```

50  compute_first()
51
52
53  # Helper for CLR
54  def first_of_sequence(sequence):
55
56      result = set()
57
58      for symbol in sequence:
59
60          if symbol not in grammar:
61              result.add(symbol)
62              return result
63
64          result |= FIRST[symbol] - {"ε"}
65
66          if "ε" not in FIRST[symbol]:
67              return result
68
69      result.add("ε")
70
71      return result
72
73
74  # ----- PRINT TABLE -----
75
76  if __name__ == "__main__":
77
78      print("\n----- FIRST TABLE ----- \n")
79
80      table = PrettyTable()
81
82      table.field_names = ["Non-Terminal", "FIRST"]
83
84      for nt in FIRST:
85
86          first_values = ", ".join(sorted(FIRST[nt]))
87
88          table.add_row([nt, first_values])
89
90      print(table)
91

```

Figure 5: Code to find FIRST

```

follow.py > ...
1  # follow.py
2
3  from cfg import grammar, start_symbol
4  from first import FIRST
5  from prettytable import PrettyTable
6
7  FOLLOW = {}
8
9  # Initialize FOLLOW sets
10 for nt in grammar:
11     FOLLOW[nt] = set()
12
13 # Start symbol gets $
14 FOLLOW[start_symbol].add("$")
15
16
17 def compute_follow():
18
19     changed = True
20
21     while changed:
22
23         changed = False
24
25         for nt in grammar:
26
27             for production in grammar[nt]:
28
29                 for i in range(len(production)):
30
31                     symbol = production[i]
32
33                     if symbol in grammar:
34
35                         # If not last symbol
36                         if i + 1 < len(production):
37
38                             next_symbol = production[i+1]
39
40                             before = FOLLOW[symbol].copy()
41
42                             if next_symbol in grammar:
43
44                                 FOLLOW[symbol] |= (
45                                     FIRST[next_symbol] - {"ε"}
46                                 )
47
48                             else:
49                                 FOLLOW[symbol].add(next_symbol)
50
51                             if before != FOLLOW[symbol]:
52                                 changed = True
53
54                     # If last symbol

```

```

55
56     else:
57
58         before = FOLLOW[symbol].copy()
59
60         FOLLOW[symbol] |= FOLLOW[nt]
61
62         if before != FOLLOW[symbol]:
63             changed = True
64
65 compute_follow()
66
67 # ----- PRINT TABLE -----
68
69 if __name__ == "__main__":
70
71     print("\n----- FOLLOW TABLE ----- \n")
72
73     table = PrettyTable()
74
75     table.field_names = ["Non-Terminal", "FOLLOW"]
76
77     for nt in FOLLOW:
78
79         follow_values = ", ".join(sorted(FOLLOW[nt]))
80
81         table.add_row([nt, follow_values])
82
83     print(table)
84

```

**Figure 6:** Code to find FOLLOW

```

dr.items.py > ...
1  # clr_items.py
2
3  from cfg import grammar
4  from first import FIRST
5
6  # Augmented grammar
7  augmented_grammar = {
8      "S'": [{"S"}]
9  }
10
11  augmented_grammar.update(grammar)
12
13
14  def first_of_sequence(sequence):
15
16      result = set()
17
18      for symbol in sequence:
19
20          if symbol not in grammar:
21              result.add(symbol)
22              return result
23
24          result |= FIRST[symbol] - {"ε"}
25
26          if "ε" not in FIRST[symbol]:
27              return result
28
29      result.add("ε")
30
31      return result
32
33
34  def closure(items):
35
36      closure_set = set(items)
37
38      while True:
39
40          new_items = set()
41
42          for lhs, rhs, dot, lookahead in closure_set:
43
44              if dot < len(rhs):
45
46                  symbol = rhs[dot]
47
48                  if symbol in augmented_grammar:
49
50                      beta = list(rhs[dot+1:])
51                      beta.append(lookahead)
52
53                      lookaheads = first_of_sequence(beta)
54

```

```

55
56
57         for prod in augmented_grammar[symbol]:
58
59             for la in lookaheads:
60
61                 item = (
62                     symbol,
63                     tuple(prod),
64                     0,
65                     la
66                 )
67
68                 if item not in closure_set:
69                     new_items.add(item)
70
71             if not new_items:
72                 break
73
74             closure_set |= new_items
75
76         return closure_set
77
78
79  def goto(items, symbol):
80
81      moved = set()
82
83      for lhs, rhs, dot, lookahead in items:
84
85          if dot < len(rhs) and rhs[dot] == symbol:
86
87              moved.add(
88                  (lhs, rhs, dot + 1, lookahead)
89              )
90
91      return closure(moved)
92
93
94  def get_symbols():
95
96      symbols = set()
97
98      for nt in augmented_grammar:
99
100          symbols.add(nt)
101
102          for prod in augmented_grammar[nt]:
103              symbols |= set(prod)
104
105      symbols.discard("ε")
106
107      return symbols

```



```

1088 def build_states():
1089
1090     start_item = closure({
1091         ("S'", tuple(["S"]), 0, "$")
1092     })
1093
1094     states = [start_item]
1095
1096     # ★ IMPORTANT FIX:
1097     # Ensure 'S' is processed FIRST
1098     symbols = sorted(
1099         get_symbols(),
1100         key=lambda x: (x != "S", x)
1101     )
1102
1103     while True:
1104
1105         new_states = []
1106
1107         for state in states:
1108
1109             for sym in symbols:
1110
1111                 g = goto(state, sym)
1112
1113                 if g and g not in states and g not in new_states:
1114                     new_states.append(g)
1115
1116             if not new_states:
1117                 break
1118
1119         states.extend(new_states)
1120
1121     return states
1122
1123
1124 if __name__ == "__main__":
1125
1126     states = build_states()
1127
1128     print("\n----- CLR(1) ITEM SETS -----")
1129
1130     for i, state in enumerate(states):
1131
1132         print(f"{i}\nI{1}:\n")
1133
1134         for item in state:
1135
1136             lhs, rhs, dot, la = item
1137
1138             rhs = list(rhs)
1139
1140             rhs.insert(dot, ".")
1141
1142             print(
1143                 lhs,
1144                 "→",
1145                 ".".join(rhs),
1146                 "\n",
1147                 la
1148             )
1149
1150
1151

```

**Figure 7:** Code to generate CLR states

```

goto_table.py > ...
1  # goto_table.py
2
3  from clr_items import build_states, goto
4  from cfg import grammar
5
6  # Build states
7  states = build_states()
8
9  # Collect all symbols
10 symbols = set()
11
12 for nt in grammar:
13     symbols.add(nt)
14
15     for prod in grammar[nt]:
16         symbols.update(prod)
17
18 # Remove epsilon
19 symbols.discard("ε")
20
21 symbols = sorted(
22     symbols,
23     key=lambda x: (x != "S", x)
24 )
25
26 print("\nGOTO TRANSITIONS\n")
27
28 for i, state in enumerate(states):
29     for symbol in symbols:
30         next_state = goto(state, symbol)
31
32         if next_state and next_state in states:
33             j = states.index(next_state)
34
35             print(f"GOTO(I{i}, {symbol}) = I{j}")
36
37
38
39
40
41

```

Figure 8: Code to create goto table

```

dr_parsing_table.py > ...
1  # dr_parsing_table.py
2
3  from clr_items import build_states, goto
4  from cfg import grammar
5  from follow import FOLLOW
6  from prettytable import PrettyTable
7
8  # Build CLR states
9  states = build_states()
10
11 # Terminals
12 terminals = [
13     "DATATYPE", "MAIN", "LPAR", "RPAR",
14     "BEGIN", "END", "WHILE",
15     "VAR", "NUMBER",
16     "COMMA", "EQ", "PLUS", "SEMI",
17     "$"
18 ]
19
20 # Non-terminals
21 non_terminals = list(grammar.keys())
22
23 # Production numbering
24 productions = []
25
26 for lhs in grammar:
27     for rhs in grammar[lhs]:
28         productions.append((lhs, rhs))
29
30 ACTION = {}
31 GOTO = {}
32
33 # Build Table
34 for i, state in enumerate(states):
35     ACTION[i] = {}
36     GOTO[i] = {}
37
38     for item in state:
39         lhs, rhs, dot, lookahead = item
40         rhs = list(rhs)
41
42         # SHIFT
43         if dot < len(rhs):
44             symbol = rhs[dot]
45             next_state = goto(state, symbol)
46
47             if next_state in states:
48                 j = states.index(next_state)
49
50                 if symbol in terminals:
51                     ACTION[i][symbol] = f"S{j}"
52                 elif symbol in non_terminals:
53                     GOTO[i][symbol] = j
54
55
56
57
58

```

```

59
60 # REDUCE / ACCEPT
61 else:
62     if lhs == "S'":
63         ACTION[i]["$"] = "ACC"
64     else:
65         prod_number = productions.index(
66             (lhs, rhs)
67         )
68         ACTION[i][lookahead] = f"R(prod_number)"
69
70
71
72
73
74 # -----
75 # COMBINED TABLE PRINT
76 # -----
77
78 print("\n----- CLR PARSING TABLE ----- \n")
79
80 table = PrettyTable()
81
82 # Combine headers
83 table.field_names = (
84     ["State"]
85     + terminals
86     + non_terminals
87 )
88
89 # Fill rows
90 for i in range(len(states)):
91     row = [i]
92
93     # ACTION part
94     for t in terminals:
95         row.append(ACTION[i].get(t, ""))
96
97     # GOTO part
98     for nt in non_terminals:
99         row.append(GOTO[i].get(nt, ""))
100
101     table.add_row(row)
102
103 print(table)
104

```

Figure 9: Code to parsing table

```

1 # validate_string.py
2
3 from lexer import tokenize
4 from clr_parsing_table import ACTION, GOTO, productions
5
6 stack = [0]
7
8 with open("input.txt") as f:
9     code = f.read()
10
11 tokens = tokenize(code)
12
13 pointer = 0
14
15 print("\nCLR STRING PARSING TRACE\n")
16
17 print("STACK\t\tINPUT\t\tACTION\n")
18
19 step = 1
20
21 while True:
22     state = stack[-1]
23     token = tokens[pointer]
24
25     print(
26         f"Step {step}:",
27         stack,
28         "\t",
29         tokens[pointer:],
30         "\t",
31         end=""
32     )
33
34     if token not in ACTION[state]:
35         print("REJECTED X")
36         break
37
38     action = ACTION[state][token]
39
40     # ----- SHIFT -----
41
42     if action.startswith("S"):
43         next_state = int(action[1:])
44         print("SHIFT", next_state)
45
46         stack.append(token)
47         stack.append(next_state)
48
49         pointer += 1
50
51     # ----- REDUCE -----
52
53     elif action.startswith("R"):
54

```

```

1 # string_trace_parser.py
2
3 from prettytable import PrettyTable
4
5
6 # ----- TOKENIZER -----
7
8 def tokenize_string(user_input):
9
10     words = user_input.split()
11
12     tokens = []
13
14     for word in words:
15
16         if word == "int":
17             tokens.append("DATATYPE")
18
19         elif word == "main":
20             tokens.append("MAIN")
21
22         elif word == "begin":
23             tokens.append("BEGIN")
24
25         elif word == "reverse":
26             tokens.append("REVERSE")
27
28         elif word == "end":
29             tokens.append("END")
30
31         else:
32             tokens.append("STRING")
33
34     tokens.append("$")
35
36     return tokens
37
38
39 # ----- PARSER -----
40
41 def parse_string(tokens):
42
43     # Expected sequence
44     pattern = [
45         "DATATYPE",
46         "MAIN",
47         "BEGIN",
48         "REVERSE",
49         "STRING",
50         "END",
51         "$"
52     ]
53
54     stack = [0]
55
56     pointer = 0
57     step = 1
58
59     table = PrettyTable()
60
61     table.field_names = [
62         "Step",
63         "Stack",
64         "Input",
65         "Action"
66     ]

```

```

87 while pointer < len(pattern):
88
89     stack_str = " ".join(map(str, stack))
90     input_str = " ".join(tokens[pointer:])
91
92     if tokens[pointer] == pattern[pointer]:
93
94         # SHIFT simulation
95         next_state = pointer + 1
96
97         table.add_row([
98             step,
99             stack_str,
100             input_str,
101             f"SHIFT {next_state}"
102         ])
103
104         stack.append(tokens[pointer])
105         stack.append(next_state)
106
107         pointer += 1
108
109     else:
110
111         table.add_row([
112             step,
113             stack_str,
114             input_str,
115             "REJECTED ❌"
116         ])
117
118         print("\n----- STRING TRACE TABLE ----- \n")
119         print(table)
120
121         return False
122
123     step += 1
124
125 # REDUCE
126 table.add_row([
127     step,
128     " ".join(map(str, stack)),
129     "$",
130     "REDUCE S → DATATYPE MAIN BEGIN REVERSE STRING END"
131 ])
132
133 step += 1
134
135 # ACCEPT
136 table.add_row([
137     step,
138     "0 S",
139     "$",
140     "ACCEPTED ✅"
141 ])
142
143 print("\n----- STRING TRACE TABLE ----- \n")
144 print(table)
145
146 return True
147
148
149

```

```

130
131 # ----- REVERSE -----
132
133 def reverse_string(user_input):
134
135     words = user_input.split()
136
137     text = words[4]
138
139     return text[::-1]
140
141
142 # ----- MAIN -----
143
144 print("\nEnter your string:\n")
145
146 user_input = input("> ")
147
148 tokens = tokenize_string(user_input)
149
150 print("\nTokens:")
151 print(tokens)
152
153 valid = parse_string(tokens)
154
155 if valid:
156
157     result = reverse_string(user_input)
158
159     print("\nReversed String:", result)
160

```

**Figure 9:** Code to validate input string

This is a simplified implementation, and a complete compiler would involve additional steps such as semantic analysis, intermediate code generation, and code optimization. Additionally, error handling and reporting would need to be enhanced for a production-quality compiler.

## RESULTS

===== TOKENS TABLE =====

| Lexeme | Token  |
|--------|--------|
| int    | INT    |
| main   | MAIN   |
| (      | LP     |
| )      | RP     |
| begin  | BEGIN  |
| n      | ID     |
| ,      | COMMA  |
| re     | ID     |
| =      | EQ     |
| 0      | NUMBER |
| rem    | ID     |
| ;      | SEMI   |
| while  | WHILE  |
| expr   | ID     |
| +      | PLUS   |
| end    | END    |

Figure 10: Token Generation

CFG Grammar:

```
S -> [['DATATYPE', 'MAIN', 'LPAR', 'RPAR', 'BEGIN', 'DECL', 'STMT', 'END']]
DECL -> [['DATATYPE', 'VAR', 'COMMA', 'VAR', 'EQ', 'NUMBER', 'COMMA', 'VAR', 'SEMI']]
STMT -> [['WHILE', 'LPAR', 'EXPR', 'RPAR', 'BEGIN', 'ASSIGN', 'END']]
ASSIGN -> [['VAR', 'EQ', 'EXPR', 'SEMI']]
EXPR -> [['VAR'], ['VAR', 'PLUS', 'VAR']]
```

Figure 13: Grammar

===== FIRST TABLE =====

| Non-Terminal | FIRST    |
|--------------|----------|
| S            | DATATYPE |
| DECL         | DATATYPE |
| STMT         | WHILE    |
| ASSIGN       | VAR      |
| EXPR         | VAR      |

Figure 16: FIRST Table

===== FOLLOW TABLE =====

| Non-Terminal | FOLLOW     |
|--------------|------------|
| S            | \$         |
| DECL         | WHILE      |
| STMT         | END        |
| ASSIGN       | END        |
| EXPR         | RPAR, SEMI |

Figure 17: FOLLOW Table

#### GOTO TRANSITIONS

```

GOTO(I0, S) = I1
GOTO(I0, DATATYPE) = I2
GOTO(I2, MAIN) = I3
GOTO(I3, LPAR) = I4
GOTO(I4, RPAR) = I5
GOTO(I5, BEGIN) = I6
GOTO(I6, DATATYPE) = I7
GOTO(I6, DECL) = I8
GOTO(I7, VAR) = I9
GOTO(I8, STMT) = I10
GOTO(I8, WHILE) = I11
GOTO(I9, COMMA) = I12
GOTO(I10, END) = I13
GOTO(I11, LPAR) = I14
GOTO(I12, VAR) = I15
GOTO(I14, EXPR) = I16
GOTO(I14, VAR) = I17
GOTO(I15, EQ) = I18
GOTO(I16, RPAR) = I19
GOTO(I17, PLUS) = I20
GOTO(I18, NUMBER) = I21
GOTO(I19, BEGIN) = I22
GOTO(I20, VAR) = I23
GOTO(I21, COMMA) = I24
GOTO(I22, ASSIGN) = I25
GOTO(I22, VAR) = I26
GOTO(I24, VAR) = I27
GOTO(I25, END) = I28
GOTO(I26, EQ) = I29
GOTO(I27, SEMI) = I30
GOTO(I29, EXPR) = I31
GOTO(I29, VAR) = I32
GOTO(I31, SEMI) = I33
GOTO(I32, PLUS) = I34
GOTO(I34, VAR) = I35

```

Figure 17: GOTO TRANSITIONS

===== CLR PARSING TABLE =====

| State | DATATYPE | MAIN | LPAR | RPAR | BEGIN | END | WHILE | VAR | NUMBER | COMMA | EQ  | PLUS | SEMI | \$  | S  | DECL | STMT | ASSIGN | EXPR |
|-------|----------|------|------|------|-------|-----|-------|-----|--------|-------|-----|------|------|-----|----|------|------|--------|------|
| 0     | S2       |      |      |      |       |     |       |     |        |       |     |      |      |     | 1  |      |      |        |      |
| 1     |          |      |      |      |       |     |       |     |        |       |     |      |      | ACC |    |      |      |        |      |
| 2     |          | S3   |      |      |       |     |       |     |        |       |     |      |      |     |    |      |      |        |      |
| 3     |          |      | S4   |      |       |     |       |     |        |       |     |      |      |     |    |      |      |        |      |
| 4     |          |      |      | S5   |       |     |       |     |        |       |     |      |      |     |    |      |      |        |      |
| 5     |          |      |      |      | S6    |     |       |     |        |       |     |      |      |     |    |      |      |        |      |
| 6     | S7       |      |      |      |       |     |       |     |        |       |     |      |      |     |    | 8    |      |        |      |
| 7     |          |      |      |      |       |     |       | S9  |        |       |     |      |      |     |    |      |      |        |      |
| 8     |          |      |      |      |       |     | S11   |     |        |       |     |      |      |     |    |      | 10   |        |      |
| 9     |          |      |      |      |       |     |       |     |        | S12   |     |      |      |     |    |      |      |        |      |
| 10    |          |      |      |      |       | S13 |       |     |        |       |     |      |      |     |    |      |      |        |      |
| 11    |          |      | S14  |      |       |     |       |     |        |       |     |      |      |     |    |      |      |        |      |
| 12    |          |      |      |      |       |     |       | S15 |        |       |     |      |      |     |    |      |      |        |      |
| 13    |          |      |      |      |       |     |       | S17 |        |       |     |      |      | R0  |    |      |      |        |      |
| 14    |          |      |      |      |       |     |       |     |        |       |     |      |      |     |    |      |      |        | 16   |
| 15    |          |      |      |      |       |     |       |     |        |       | S18 |      |      |     |    |      |      |        |      |
| 16    |          |      |      | S19  |       |     |       |     |        |       |     |      |      |     |    |      |      |        |      |
| 17    |          |      |      | R4   |       |     |       |     |        |       |     | S20  |      |     |    |      |      |        |      |
| 18    |          |      |      |      |       |     |       |     | S21    |       |     |      |      |     |    |      |      |        |      |
| 19    |          |      |      |      | S22   |     |       |     |        |       |     |      |      |     |    |      |      |        |      |
| 20    |          |      |      |      |       |     |       |     | S23    |       |     |      |      |     |    |      |      |        |      |
| 21    |          |      |      |      |       |     |       |     |        | S24   |     |      |      |     |    |      |      | 25     |      |
| 22    |          |      |      |      |       |     |       |     | S26    |       |     |      |      |     |    |      |      |        |      |
| 23    |          |      |      | R5   |       |     |       |     |        |       |     |      |      |     |    |      |      |        |      |
| 24    |          |      |      |      |       |     |       |     | S27    |       |     |      |      |     |    |      |      |        |      |
| 25    |          |      |      |      |       | S28 |       |     |        |       |     |      |      |     |    |      |      |        |      |
| 26    |          |      |      |      |       |     |       |     |        |       | S29 |      |      |     |    |      |      |        |      |
| 27    |          |      |      |      |       |     |       |     |        |       |     |      |      |     |    | S30  |      |        |      |
| 28    |          |      |      |      |       | R2  |       |     |        |       |     |      |      |     |    |      |      |        |      |
| 29    |          |      |      |      |       |     |       | S32 |        |       |     |      |      |     |    |      |      |        | 31   |
| 30    |          |      |      |      |       |     | R1    |     |        |       |     |      |      |     |    |      |      |        |      |
| 31    |          |      |      |      |       |     |       |     |        |       |     |      |      |     |    |      |      |        |      |
| 32    |          |      |      |      |       |     |       |     |        |       |     |      |      | S33 | R4 |      |      |        |      |
| 33    |          |      |      |      |       | R3  |       |     |        |       |     | S34  |      |     |    |      |      |        |      |
| 34    |          |      |      |      |       |     |       | S35 |        |       |     |      |      |     |    |      |      |        |      |
| 35    |          |      |      |      |       |     |       |     |        |       |     |      |      | R5  |    |      |      |        |      |

Figure 18: CLR parsing Table

PS C:\Users\USER\Desktop\Final try> python string\_trace\_parser.py

===== STRING TRACE TABLE =====

| Step | Stack                                                     | Input                                     | Action                                            |
|------|-----------------------------------------------------------|-------------------------------------------|---------------------------------------------------|
| 1    | 0                                                         | DATATYPE MAIN BEGIN REVERSE STRING END \$ | SHIFT 1                                           |
| 2    | 0 DATATYPE 1                                              | MAIN BEGIN REVERSE STRING END \$          | SHIFT 2                                           |
| 3    | 0 DATATYPE 1 MAIN 2                                       | BEGIN REVERSE STRING END \$               | SHIFT 3                                           |
| 4    | 0 DATATYPE 1 MAIN 2 BEGIN 3                               | REVERSE STRING END \$                     | SHIFT 4                                           |
| 5    | 0 DATATYPE 1 MAIN 2 BEGIN 3 REVERSE 4                     | STRING END \$                             | SHIFT 5                                           |
| 6    | 0 DATATYPE 1 MAIN 2 BEGIN 3 REVERSE 4 STRING 5            | END \$                                    | SHIFT 6                                           |
| 7    | 0 DATATYPE 1 MAIN 2 BEGIN 3 REVERSE 4 STRING 5 END 6      | \$                                        | SHIFT 7                                           |
| 8    | 0 DATATYPE 1 MAIN 2 BEGIN 3 REVERSE 4 STRING 5 END 6 \$ 7 | \$                                        | REDUCE S → DATATYPE MAIN BEGIN REVERSE STRING END |
| 9    | 0 S                                                       | \$                                        | ACCEPTED ✓                                        |

Reversed String: olleh

PS C:\Users\USER\Desktop\Final try> python string\_trace\_parser.py

Enter your string:

> int main begin reverse hello

Tokens:  
['DATATYPE', 'MAIN', 'BEGIN', 'REVERSE', 'STRING', '\$']

===== STRING TRACE TABLE =====

| Step | Stack                                          | Input                                 | Action     |
|------|------------------------------------------------|---------------------------------------|------------|
| 1    | 0                                              | DATATYPE MAIN BEGIN REVERSE STRING \$ | SHIFT 1    |
| 2    | 0 DATATYPE 1                                   | MAIN BEGIN REVERSE STRING \$          | SHIFT 2    |
| 3    | 0 DATATYPE 1 MAIN 2                            | BEGIN REVERSE STRING \$               | SHIFT 3    |
| 4    | 0 DATATYPE 1 MAIN 2 BEGIN 3                    | REVERSE STRING \$                     | SHIFT 4    |
| 5    | 0 DATATYPE 1 MAIN 2 BEGIN 3 REVERSE 4          | STRING \$                             | SHIFT 5    |
| 6    | 0 DATATYPE 1 MAIN 2 BEGIN 3 REVERSE 4 STRING 5 | \$                                    | REJECTED ✗ |

Figure 19: String Validation



## CONCLUSION

**Successful Implementation:** The provided Python script successfully implements a compiler for a subset of the programming language. It includes a lexical analyzer that identifies tokens such as keywords, identifiers, operators, and delimiters, as well as a syntax analyzer that verifies the structure of the input code. **Lexical and Syntax Analysis:** The compiler performs lexical analysis by tokenizing the input code, and recognizing various language constructs. The syntax analyzer checks if the sequence of tokens adheres to the defined grammar rules, capturing the high-level structure of the source code. **CLR Parsing Table:** The CLR (1) parsing table is generated based on the grammar, FIRST, and FOLLOW sets. This table is crucial for the parsing process, providing a systematic approach to validate the input string against the grammar. **Methodology and Limitations:** The methodology involves lexical analysis, syntax analysis, and CLR parsing table generation. However, this implementation is simplified and lacks advanced features such as semantic analysis, optimization, and error recovery. It also assumes a specific grammar and may not cover all language features. **Room for Improvement:** To enhance the compiler's functionality, future work could focus on incorporating additional language features, improving error handling, and extending support for more complex input programs. This project lays the foundation for understanding compiler design concepts and can be extended for more comprehensive language support.

## REFERENCES

- [1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd Edition). Pearson.
- [2] Cooper, K., & Torczon, L. (2011). Engineering a Compiler (2nd Edition). Morgan Kaufmann.
- [3] <https://www.javatpoint.com/compiler-tutorial>
- [4] [https://www.w3schools.com/python/python\\_regex.asp](https://www.w3schools.com/python/python_regex.asp)
- [5] <https://ply.readthedocs.io/en/latest/>
- [6] <https://www.geeksforgeeks.org/creating-tables-with-prettytable-library-python/>